```
def generate_spike_stream(patterns,
batch_size=32, time_steps=50,
sync_prob=0.3):
    """Генерация реалистичного
асинхронного потока спайков"""
    N = patterns.shape[1]

    streams = []
    for _ in range(batch_size):
        # Базовый паттерн + шум
        base_pattern = patterns[torch.randint(0,
len(patterns), (1,))]
        stream = torch.zeros(time_steps, 3 * N)
# Триарный: {-1,0,+1}

        for t in range(time_steps):
            # С вероятностью sync_prob -
синхронизированный спайк
            if torch.rand(1) < sync_prob:
                spike = base_pattern + 0.5 *
torch.randn_like(base_pattern)
            else:
```

```
                spike =
torch.zeros_like(base_pattern)

            # Триарное кодирование
            stream[t, :N] = (spike > 0).float() * 2 - 1
# +1/-1
            stream[t, N:2*N] = (spike.abs() >
0.5).float()  # Интенсивность
            stream[t, 2*N:] = torch.zeros(N)  #
Silence channel


streams.append(stream.reshape(time_steps,
-1))

    return torch.stack(streams)  # [batch, time,
3N]
```